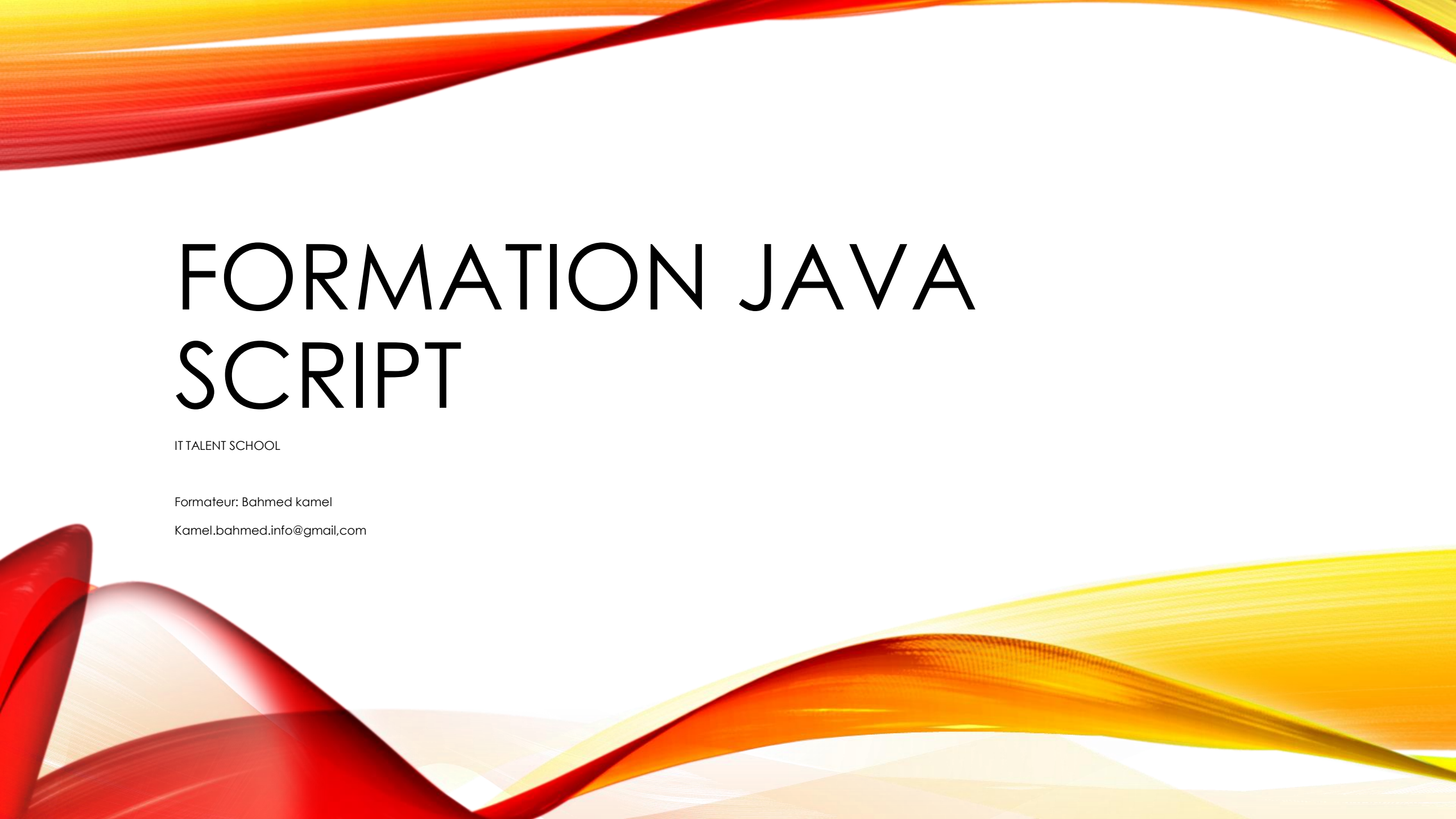




FORMATION JAVA SCRIPT

IT TALENT SCHOOL

Formateur: Bahmed kamel

Kamel.bahmed.info@gmail.com



PRÉREQUIS

1. Connaissances générales de la terminologie de l'algorithmique objet (type, fonction, etc.)
2. HTML
3. CSS

PROGRAMME

Les base je javascript

- Introduction
- Possibilités d'affichage
- Variables
- Let
- Const
- Opérateurs arithmétiques
- Opérateurs d'affectation
- Opérateurs de chaîne
- Opérateurs de comparaison
- Opérateurs logiques
- Fonctions
- Événements
- Méthodes de chaîne
- Tableaux
- Les conditions
- Les boucles

• Manipulation du DOM (Document Object Model)

- BOM
- Événements de synchronisation
- Méthodes HTML DOM
- Recherche d'éléments
- Modification des éléments HTML
- Ajout et suppression d'éléments
- Recherche d'objets HTML
- Formulaire JavaScript
- Modification du style HTML
- Animation JavaScript
- Écouteur d'événements
- Navigation JavaScript
- Relations de nœud
- Naviguer entre les nœuds
- Ajout et suppression de nœuds
- L'objet HTML Collection
- JQuery (Extra)

JAVASCRIPT

- Le JavaScript est un langage de programmation créé en 1995. Le JavaScript est aujourd'hui l'un des langages de programmation les plus populaires et il fait partie des langages web dits « standards » avec le HTML et le CSS. Son évolution est gérée par le groupe ECMA International qui se charge de publier les standards de ce langage.
- On dit que le HTML, le CSS et le JavaScript sont des standards du web car les principaux navigateurs web (Google Chrome, Safari, Firefox, etc.) savent tous « lire » (ou « comprendre » ou « interpréter ») ces langages et les interprètent généralement de la même façon ce qui signifie qu'un même code va généralement produire le même résultat dans chaque navigateur.

JAVASCRIPT

- Pour définir ce qu'est le JavaScript et le situer par rapport aux autres langages, et donc pour comprendre les intérêts et usages du JavaScript il faut savoir que :

langage dynamique

- c'est-à-dire un langage qui va nous permettre de générer du contenu dynamique pour nos pages web

langage (principalement)
côté client

- Un langage « côté client » ou « client side » est un langage qui va être exécuté dans le navigateur des utilisateurs qui demandent la page..

langage interprété

- Cela signifie qu'il va pouvoir être exécuté directement sous réserve qu'on possède le logiciel interpréteur. tous les navigateurs connus possèdent leur interpréteur JavaScript.

langage orienté objet

- Le JavaScript est un langage qui possède un fort potentiel pour la programmation orientée objet (abrégée en POO).

OÙ ÉCRIRE LE CODE JAVASCRIPT ?

1. La balise <script>

- En HTML, le code JavaScript est inséré entre les balises <script> et </script>

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript in Body</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "My First
JavaScript";
</script>

</body>
</html>
```

JavaScript in Body

My First JavaScript

Vous pouvez placer n'importe quel nombre de scripts dans un document HTML.

Les scripts peuvent être placés dans le <body>, ou dans la <head>section d'une page HTML, ou dans les deux.

- 2. JavaScript externe
- Les scripts peuvent également être placés dans des fichiers externes :

Fichier externe : monScript.js

```
function myFunction() {  
    document.getElementById("demo").innerHTML = "Paragraph changed.";  
}
```

```
<!DOCTYPE html>  
<html>  
<body>  
<h2>Demo External JavaScript</h2>  
<p id="demo">A Paragraph.</p>  
<button type="button" onclick="myFunction()">Try it</button>  
<script src="myScript.js"></script>  
</body>  
</html>
```

Placer des scripts dans des fichiers externes présente certains avantages :

- Il sépare HTML et code
- Il rend HTML et JavaScript plus faciles à lire et à maintenir
- Les fichiers JavaScript mis en cache peuvent accélérer le chargement des pages

Demo External JavaScript

A Paragraph.

Try it

RÉFÉRENCES EXTERNES

- Un script externe peut être référencé de 3 manières différentes :
 - Avec une URL complète (une adresse Web complète)
 - Avec un chemin de fichier (comme /js/)
 - Sans aucun chemin (js et html localiser dans le même dossier)
- Pour ajouter plusieurs fichiers de script à une page, utilisez plusieurs balises de script :

Exemple

```
<script src="myScript1.js"></script>  
<script src="myScript2.js"></script>
```


POSSIBILITÉS D'AFFICHAGE JAVASCRIPT

- JavaScript peut "afficher" les données de différentes manières :
- Écrire dans un élément HTML, en utilisant `innerHTML`.
- Écrire dans la sortie HTML à l'aide de `document.write()`.
- Écrire dans une boîte d'alerte, en utilisant `window.alert()`.
- Écrire dans la console du navigateur, en utilisant `console.log()`.

UTILISER INNERHTML

- Pour accéder à un élément HTML, JavaScript peut utiliser la `document.getElementById(id)` méthode.
- L' `id` attribut définit l'élément HTML. La `innerHTML` propriété définit le contenu HTML :

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My First Paragraph.</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>

</body>
</html>
```

My First Web Page

My First Paragraph.

11|

UTILISATION DE DOCUMENT.WRITE()

- A des fins de test, il est pratique d'utiliser document.write():

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<p>Never call document.write after the document has finished loading.
It will overwrite the whole document.</p>

<script>
document.write(5 + 6);
</script>

</body>
</html>
```

My First Web Page

My first paragraph.

Never call document.write after the document has finished loading. It will overwrite the whole document.



UTILISATION DE WINDOW.ALERT()

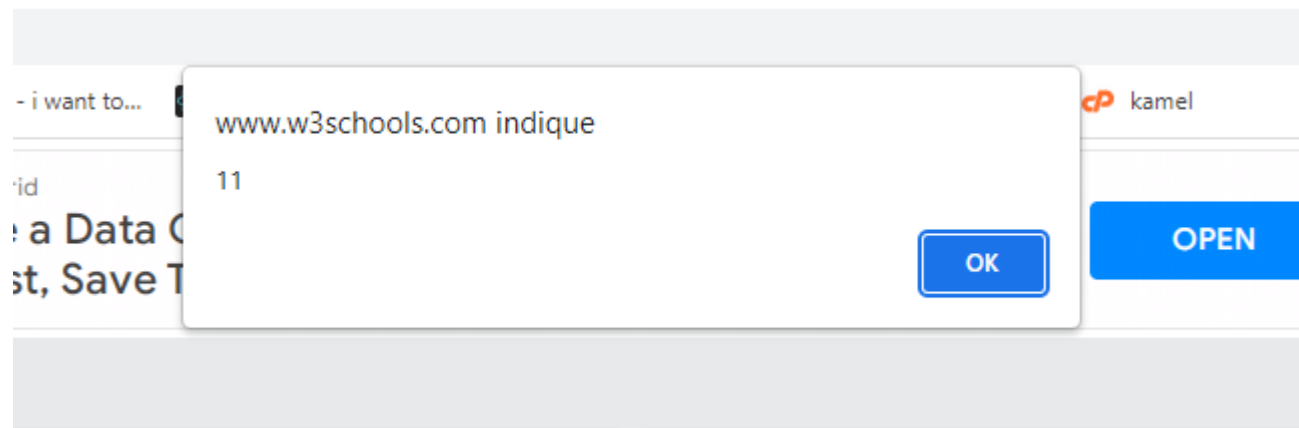
- Vous pouvez utiliser une boîte d'alerte pour afficher les données :

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html>
```



UTILISATION DE CONSOLE.LOG()

- À des fins de débogage, vous pouvez appeler la console.log() méthode dans le navigateur pour afficher les données.

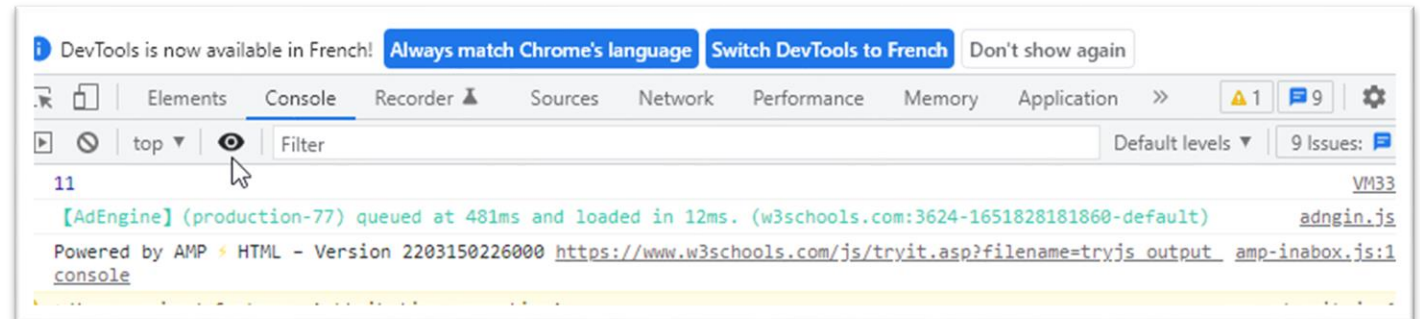
```
<!DOCTYPE html>
<html>
<body>

<h2>Activate Debugging</h2>

<p>F12 on your keyboard will activate debugging.</p>
<p>Then select "Console" in the debugger menu.</p>
<p>Then click Run again.</p>

<script>
console.log(5 + 6);
</script>

</body>
</html>
```



VARIABLES JAVASCRIPT

- Les variables sont des conteneurs pour stocker des données (stocker des valeurs de données).
- Vous déclarez une variable JavaScript avec le var ou le let mot clé :

```
var carName;
```

ou alors:

```
let carName;
```

- Après la déclaration, la variable n'a pas de valeur (techniquement c'est undefined).
- Pour **affecter** une valeur à la variable, utilisez le signe égal :

```
carName = "Volvo";
```

- Vous pouvez également affecter une valeur à la variable lorsque vous la déclarez :

```
let carName = "Volvo";
```

```
<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>
```

Dans l'exemple ci-dessous, nous créons une variable appelée carName et lui attribuons la valeur "Volvo".

Ensuite, nous "produisons" la valeur dans un paragraphe HTML avec id="demo":

LET JAVASCRIPT

- Les variables définies avec let ne peuvent pas être redéclarées .
- Vous ne pouvez pas redéclarer accidentellement une variable.
- Avec let vous ne pouvez pas faire ceci :
- Par contre avec var vous pouvez le faire

```
let x = "John Doe";  
  
let x = 0;  
  
// SyntaxError: 'x' has already been declared
```

- Les variables déclarées à l'intérieur d'un bloc { } avec Let ne sont pas accessibles depuis l'extérieur du bloc :

Exemple

```
{  
  var x = 2;  
}  
  
// x CAN be used here
```

Exemple

```
{  
  let x = 2;  
}  
  
// x can NOT be used here
```

CONST JAVASCRIPT

- Une const variable ne peut pas être réaffectée :

Exemple

```
const PI = 3.141592653589793;  
PI = 3.14;           // This will give an error  
PI = PI + 10;        // This will also give an error
```

Incorrect

```
const PI;  
PI = 3.14159265359;
```

OPÉRATEURS ARITHMÉTIQUES JAVASCRIPT

- Les opérateurs arithmétiques sont utilisés pour effectuer des opérations arithmétiques sur des nombres :

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

OPÉRATEURS D'AFFECTATION JAVASCRIPT

- Les opérateurs d'affectation attribuent des valeurs aux variables JavaScript.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

OPÉRATEURS DE CHAÎNE JAVASCRIPT

- L' +opérateur peut également être utilisé pour ajouter (concaténer) des chaînes.

Exemple

```
let text1 = "John";  
let text2 = "Doe";  
let text3 = text1 + " " + text2;
```

Le résultat de text3 sera :

John Doe

Exemple

```
let x = 5 + 5;  
let y = "5" + 5;  
let z = "Hello" + 5;
```

Le résultat de x , y et z sera :

10
55
Hello5

L'ajout de deux nombres renverra la somme, mais l'ajout d'un nombre et d'une chaîne renverra une chaîne :

OPÉRATEURS DE COMPARAISON JAVASCRIPT

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

OPÉRATEURS LOGIQUES JAVASCRIPT

Operator	Description
&&	logical and
	logical or
!	logical not

FONCTIONS JAVASCRIPT

- Une fonction JavaScript est un bloc de code conçu pour effectuer une tâche particulière.
- Une fonction JavaScript est exécutée lorsque "quelque chose" l'invoque (l'appelle).

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>

<p>This example calls a function which performs a calculation and returns the result:</p>

<p id="demo"></p>

<script>
var x = myFunction(4, 3);
document.getElementById("demo").innerHTML = x;

function myFunction(a, b) {
  return a * b;
}
</script>

</body>
</html>
```

JavaScript Functions

This example calls a function which performs a calculation and returns the result:

12|

VARIABLES LOCALES

- Les variables déclarées dans une fonction JavaScript deviennent **LOCALES** à la fonction.
- Les variables locales ne sont accessibles qu'à partir de la fonction.

Exemple

```
// code here can NOT use carName

function myFunction() {
  let carName = "Volvo";
  // code here CAN use carName
}

// code here can NOT use carName
```

- Étant donné que les variables locales ne sont reconnues qu'à l'intérieur de leurs fonctions, les variables portant le même nom peuvent être utilisées dans différentes fonctions.
- Les variables locales sont créées lorsqu'une fonction démarre et supprimées lorsque la fonction est terminée.

ÉVÉNEMENTS JAVASCRIPT

- Les événements HTML sont **des "choses"** qui arrivent aux éléments HTML.
- Lorsque JavaScript est utilisé dans des pages HTML, JavaScript peut **"réagir"** à ces événements.
- Voici quelques exemples d'événements HTML :
 - Une page Web HTML a fini de se charger
 - Un champ de saisie HTML a été modifié
 - Un bouton HTML a été cliqué
- HTML permet d'ajouter des attributs de gestionnaire d'événements, **avec du code JavaScript**, aux éléments HTML.

Avec guillemets simples :

```
<element event='some JavaScript'>
```

Avec guillemets doubles :

```
<element event="some JavaScript">
```



```
<!DOCTYPE html>
<html>

<body>

    <button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>

    <p id="demo"></p>

</body>

</html>
```

The time is?

Mon May 09 2022 08:59:20 GMT+0100 (heure normale d'Afrique de l'Ouest)

```
<button onclick="displayDate()">The time is?</button>

<script>
function displayDate() {
    document.getElementById("demo").innerHTML = Date();
}
</script>
```

Le code JavaScript fait souvent plusieurs lignes. Il est plus courant de voir des attributs d'événement appeler des fonctions :

ÉVÉNEMENTS HTML COURANTS

Événement	La description
onchange	Un élément HTML a été modifié
onclick	L'utilisateur clique sur un élément HTML
onmouseover	L'utilisateur déplace la souris sur un élément HTML
onmouseout	L'utilisateur éloigne la souris d'un élément HTML
onkeydown	L'utilisateur appuie sur une touche du clavier
onload	Le navigateur a fini de charger la page

MÉTHODES DE CHAÎNE JAVASCRIPT

- La **length** propriété renvoie la longueur d'une chaîne :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript String Properties</h2>

<p>The length property returns the length of a string:</p>

<p id="demo"></p>

<script>
let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
document.getElementById("demo").innerHTML = text.length;
</script>

</body>
</html>
```

JavaScript String Properties

The length property returns the length of a string:

26

Method	Description
<u>charAt()</u>	Renvoie le caractère à un index spécifié (position)
<u>charCodeAt()</u>	Renvoie l'unicode du caractère à un index spécifié
<u>concat()</u>	Retourne deux chaînes rejoints ou plus
<u>endsWith()</u>	Renvoie si une chaîne se termine par une valeur spécifiée
<u>fromCharCode()</u>	Renvoie les valeurs Unicode en tant que caractères
<u>includes()</u>	Renvoie si une chaîne contient une valeur spécifiée
<u>indexOf()</u>	Renvoie l'index (position) de la première occurrence d'une valeur dans une chaîne
<u>lastIndexOf()</u>	Renvoie l'index (position) de la dernière occurrence d'une valeur dans une chaîne
<u>localeCompare()</u>	Compare deux chaînes dans le lieu actuel
<u>match()</u>	Recherche une chaîne pour une valeur, ou une expression régulière, et renvoie les correspondances
<u>repeat()</u>	Renvoie une nouvelle chaîne avec un certain nombre de copies d'une chaîne

<u>replace()</u>	Recherche une chaîne pour une valeur, ou une expression régulière, et renvoie une chaîne où les valeurs sont remplacées
<u>search()</u>	Recherche une chaîne pour une valeur ou une expression régulière, et renvoie l'index (position) de la correspondance
<u>slice()</u>	Extrait une partie d'une chaîne et renvoie une nouvelle chaîne
<u>split()</u>	Divise une chaîne dans un tableau de sous-chaînes
<u>startsWith()</u>	Vérifie si une chaîne commence par des caractères spécifiés
<u>substr()</u>	Extrait un certain nombre de caractères d'une chaîne, à partir d'un index de départ (position)
<u>substring()</u>	Extrait les caractères d'une chaîne, entre deux indices spécifiés (positions)
<u>toLocaleLowerCase()</u>	Renvoie une chaîne convertie en lettres minuscules, en utilisant le lieu de l'hôte
<u>toLocaleUpperCase()</u>	Renvoie une chaîne convertie en lettres majuscules, en utilisant le lieu de l'hôte
<u>toLowerCase()</u>	Renvoie une chaîne convertie en lettres minuscules
<u>toString()</u>	Renvoie une chaîne ou un objet de chaîne en tant que chaîne
<u>toUpperCase()</u>	Renvoie une chaîne convertie en lettres majuscules
<u>trim()</u>	Renvoie une chaîne avec des espaces blancs supprimés
<u>valueOf()</u>	Renvoie la valeur primitive d'une chaîne ou d'un objet de chaîne

TABLEAUX JAVASCRIPT

- Un tableau est une variable spéciale, qui peut contenir plusieurs valeurs :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Arrays</h2>

<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

JavaScript Arrays

Saab,Volvo,BMW

Utilisation du mot-clé JavaScript new

L'exemple suivant crée également un tableau et lui attribue des valeurs :

Exemple

```
const cars = new Array("Saab", "Volvo", "BMW");
```

- On accède à un élément du tableau en se référant au **numéro d'index** :

```
const cars = ["Saab", "Volvo", "BMW"];  
let car = cars[0];
```

Remarque : Les index de tableau commencent par 0. [0] est le premier élément. [1] est le deuxième élément.

- Cette instruction change la valeur du premier élément dans cars:

```
cars[0] = "Opel";
```

- La propriété de longueur

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
let length = fruits.length;
```

MÉTHODES DE TABLEAU JAVASCRIPT

Method	Description
<u>concat()</u>	Rejoint deux tableaux ou plus et renvoie une copie des tableaux joints
<u>copyWithin()</u>	Copie les éléments du tableau dans le tableau, vers et depuis des positions spécifiées
<u>entries()</u>	Renvoie un objet d'itération du tableau de paire de clés / valeur
<u>every()</u>	Vérifie si chaque élément d'un tableau passe un test
<u>fill()</u>	Remplissez les éléments dans un tableau avec une valeur statique
<u>filter()</u>	Crée un nouveau tableau avec chaque élément d'un tableau qui passe un test
<u>find()</u>	Renvoie la valeur du premier élément d'un tableau qui passe un test
<u>findIndex()</u>	Renvoie l'index du premier élément d'un tableau qui passe un test
<u>forEach()</u>	Appelle une fonction pour chaque élément de tableau
<u>from()</u>	Crée un tableau à partir d'un objet
<u>includes()</u>	Vérifiez si un tableau contient l'élément spécifié
<u>indexOf()</u>	Recherchez le tableau un élément et renvoie sa position
<u>isArray()</u>	Vérifie si un objet est un tableau
<u>join()</u>	Rejoint tous les éléments d'un tableau dans une chaîne
<u>keys()</u>	Renvoie un objet d'itération du tableau, contenant les clés du tableau d'origine
<u>lastIndexOf()</u>	Recherchez le tableau un élément, à partir de la fin, et renvoie sa position
<u>map()</u>	Crée un nouveau tableau avec le résultat de l'appel d'une fonction pour chaque élément de tableau
<u>pop()</u>	Supprime le dernier élément d'un tableau et renvoie cet élément
<u>push()</u>	Ajoute de nouveaux éléments à la fin d'un tableau et renvoie la nouvelle longueur
<u>unshift()</u>	Ajoute de nouveaux éléments au début d'un tableau et renvoie la nouvelle longueur
<u>valueOf()</u>	Renvoie la valeur primitive d'un tableau

<u>reduce()</u>	Réduire les valeurs d'un tableau à une seule valeur (aller à droite)
<u>reduceRight()</u>	Réduire les valeurs d'un tableau à une seule valeur (aller droit à gauche)
<u>reverse()</u>	Inverse l'ordre des éléments dans un tableau
<u>shift()</u>	Supprime le premier élément d'un tableau et renvoie cet élément
<u>slice()</u>	Sélectionne une partie d'un tableau et renvoie le nouveau tableau
<u>some()</u>	Vérifie si l'un des éléments d'un tableau passe un test
<u>sort()</u>	Trie les éléments d'un tableau
<u>splice()</u>	Ajoute / supprime les éléments d'un tableau
<u>toString()</u>	Convertit un tableau en une chaîne et renvoie le résultat

MAP DE TABLEAU JAVASCRIPT()

- La map() méthode crée un nouveau tableau en exécutant une fonction sur chaque élément du tableau.
- La map() méthode n'exécute pas la fonction pour les éléments de tableau sans valeurs.
- La map() méthode ne modifie pas le tableau d'origine.

```
<!DOCTYPE html>
<html>

<body>

  <h2>JavaScript Array.map()</h2>
  <p id="demo"></p>

  <script>
    const numbers1 = [45, 4, 9, 16, 25];
    const numbers2 = numbers1.map(myFunction);

    document.getElementById("demo").innerHTML = numbers2;

    function myFunction(value, index, array) {
      return value * 2;
    }
  </script>

</body>

</html>
```

Cet exemple multiplie chaque valeur de tableau par 2 :

JavaScript Array.map()

Creates a new array by performing a function on each array element.

90,8,18,32,50|

Notez que la fonction prend 3 arguments :

- La valeur de l'article
- L'index des articles
- Le tableau lui-même

- Lorsqu'une fonction de rappel utilise uniquement le paramètre value, les paramètres index et array peuvent être omis :

```
const numbers1 = [45, 4, 9, 16, 25];  
const numbers2 = numbers1.map(myFunction);  
  
function myFunction(value) {  
  return value * 2;  
}
```

FILTRE DE TABLEAU JAVASCRIPT()

- La filter() méthode crée un nouveau tableau avec des éléments de tableau qui réussit un test.
- Cet exemple crée un nouveau tableau à partir d'éléments dont la valeur est supérieure à 18 :

```
const numbers = [45, 4, 9, 16, 25];  
const over18 = numbers.filter(myFunction);  
  
function myFunction(value, index, array) {  
  return value > 18;  
}
```

Notez que la fonction prend 3 arguments :

- La valeur de l'article
- L'index des articles
- Le tableau lui-même

- Dans l'exemple ci-dessus, la fonction de rappel n'utilise pas les paramètres d'index et de tableau, ils peuvent donc être omis :

```
const numbers = [45, 4, 9, 16, 25];  
const over18 = numbers.filter(myFunction);  
  
function myFunction(value) {  
  return value > 18;  
}
```

L'INSTRUCTION IF

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript if</h2>

<p>Display "Good day!" if the hour is less than 18:00:</p>

<p id="demo">Good Evening!</p>

<script>
if (new Date().getHours() < 18) {
    document.getElementById("demo").innerHTML = "Good day!";
}
</script>

</body>
</html>
```

JavaScript if

Display "Good day!" if the hour is less than 18:00:

Good day!



```
<script>
const hour = new Date().getHours();
let greeting;

if (hour < 18) {
    greeting = "Good day";
} else {
    greeting = "Good evening";
}

document.getElementById("demo").innerHTML = greeting;
</script>
```

```
<script>
    const time = new Date().getHours();
    let greeting;
    if (time < 10) {
        greeting = "Good morning";
    } else if (time < 20) {
        greeting = "Good day";
    } else {
        greeting = "Good evening";
    }
    document.getElementById("demo").innerHTML = greeting;
</script>
```

BOUCLES JAVASCRIPT

Au lieu d'écrire :

```
text += cars[0] + "<br>";  
text += cars[1] + "<br>";  
text += cars[2] + "<br>";  
text += cars[3] + "<br>";  
text += cars[4] + "<br>";  
text += cars[5] + "<br>";
```

Tu peux écrire:

```
for (let i = 0; i < cars.length; i++) {  
  text += cars[i] + "<br>";  
}
```

Example

```
while (i < 10) {  
  text += "The number is " + i;  
  i++;  
}
```

LE MODÈLE D'OBJET DE NAVIGATEUR (BOM)

- Le Browser Object Model (BOM) permet à JavaScript de "parler" au navigateur.
- Il n'existe pas de normes officielles pour le Browser **O bject** Model (**BOM**).
- Étant donné que les navigateurs modernes ont implémenté (presque) les mêmes méthodes et propriétés pour l'interactivité JavaScript, on y fait souvent référence en tant que méthodes et propriétés du BOM.

WINDOW OBJET

- L' window objet est pris en charge par tous les navigateurs. Il représente la fenêtre du navigateur.
- Tous les objets, fonctions et variables JavaScript globaux deviennent automatiquement membres de l'objet window.
- Les variables globales sont des propriétés de l'objet window.
- Les fonctions globales sont des méthodes de l'objet window.
- Même l'objet document (du DOM HTML) est une propriété de l'objet window :
- `window.document.getElementById("header");`
- est le même que:
- `document.getElementById("header");`

PROPRIÉTÉS DE L'OBJET WINDOW

Property	Description
closed	Renvoie un Boolean True si une fenêtre est fermée.
console	Renvoie l'objet console pour la fenêtre. Voir aussi l'objet Console.
defaultStatus	Déprécié.
document	Renvoie l'objet document pour la fenêtre. Voir aussi l'objet de document.
frameElement	Renvoie le cadre dans lequel la fenêtre s'exécute.
frames	Renvoie tous les objets de fenêtre en cours d'exécution dans la fenêtre.
history	Renvoie l'objet historique de la fenêtre. Voir aussi l'objet historique.
innerHeight	Renvoie la hauteur de la zone de contenu de la fenêtre (fenêtre), y compris les barres de défilement
innerWidth	Renvoie la largeur de la zone de contenu d'une fenêtre (fenêtre), y compris les barres de défilement
length	Renvoie le nombre d'éléments <frame> dans la fenêtre actuelle
localStorage	Permet d'enregistrer les paires de clés / valeur dans un navigateur Web. Stocke les données sans date d'expiration
location	Renvoie l'objet d'emplacement pour la fenêtre. Voir aussi l'objet de localisation.
name	Définit ou renvoie le nom d'une fenêtre
navigator	Renvoie l'objet Navigator pour la fenêtre. Voir aussi l'objet Navigator.
opener	Renvoie une référence à la fenêtre qui a créé la fenêtre
outerHeight	Renvoie la hauteur de la fenêtre du navigateur, y compris les barres d'outils / barres de défilement
outerWidth	Renvoie la largeur de la fenêtre du navigateur, y compris les barres d'outils / barres de défilement
pageXOffset	Renvoie les pixels que le document actuel a été défilé (horizontalement) du coin supérieur gauche de la fenêtre
pageYOffset	Renvoie les pixels que le document actuel a été défilé (verticalement) du coin supérieur gauche de la fenêtre
parent	Renvoie la fenêtre parent de la fenêtre actuelle

screen	Renvoie l'objet d'écran pour la fenêtre Voir aussi l'objet d'écran
screenLeft	Renvoie la coordonnée horizontale de la fenêtre par rapport à l'écran
screenTop	Renvoie la coordonnée verticale de la fenêtre par rapport à l'écran
screenX	Renvoie la coordonnée horizontale de la fenêtre par rapport à l'écran
screenY	Renvoie la coordonnée verticale de la fenêtre par rapport à l'écran
sessionStorage	Permet d'enregistrer les paires de clés / valeur dans un navigateur Web. Stocke les données pour une session
scrollX	Un alias de PageXoffset
scrollY	Un alias de PageYoffset
self	Renvoie la fenêtre actuelle
status	Déprécié. Évitez de l'utiliser.
top	Renvoie la fenêtre du navigateur supérieur

MÉTHODES D'OBJET DE FENÊTRE

Method	Description
alert()	Affiche une boîte d'alerte avec un message et un bouton OK
atob()	Décode une chaîne codée de base-64
blur()	Supprime la mise au point de la fenêtre actuelle
btoa()	Code pour une chaîne dans la base-64
clearInterval()	Efface un jeu de minuterie avec setInterval ()
clearTimeout()	Efface un jeu de minuterie avec setTimeout ()
close()	Ferme la fenêtre actuelle
confirm()	Affiche une boîte de dialogue avec un message et un bouton OK et Annuler
focus()	Définit l'accent sur la fenêtre actuelle
getComputedStyle()	Obtient les styles CSS calculés actuels appliqués à un élément
getSelection()	Renvoie un objet de sélection représentant la plage de texte sélectionné par l'utilisateur
matchMedia()	Renvoie un objet MediaQueryList représentant la chaîne de requête multimédia CSS spécifiée
moveBy()	Déplace une fenêtre par rapport à sa position actuelle
moveTo()	Déplace une fenêtre sur la position spécifiée
open()	Ouvre une nouvelle fenêtre de navigateur

<u>print()</u>	Imprime le contenu de la fenêtre actuelle
<u>prompt()</u>	Affiche une boîte de dialogue qui invite le visiteur à l'entrée
<code>requestAnimationFrame()</code>	Demande au navigateur d'appeler une fonction pour mettre à jour une animation avant la prochaine refection
<u>resizeBy()</u>	Redimensionne la fenêtre par les pixels spécifiés
<u>resizeTo()</u>	Redimensionne la fenêtre sur la largeur et la hauteur spécifiées
<code>scroll()</code>	Déprécié. Cette méthode a été remplacée par la méthode <code>ScrollTo ()</code> .
<u>scrollBy()</u>	Défile le document par le nombre spécifié de pixels
<u>scrollTo()</u>	Défile le document aux coordonnées spécifiées
<u>setInterval()</u>	Appelle une fonction ou évalue une expression à intervalles spécifiés (en millisecondes)
<u>setTimeout()</u>	Appelle une fonction ou évalue une expression après un nombre spécifié de millisecondes
<u>stop()</u>	Empêche la fenêtre de charger

ÉVÉNEMENTS DE SYNCHRONISATION JAVASCRIPT

- L' window objet permet l'exécution de code à des intervalles de temps spécifiés.
- Ces intervalles de temps sont appelés événements de synchronisation.
- Les deux principales méthodes à utiliser avec JavaScript sont :
- `setTimeout(function, milliseconds)`
- Exécute une fonction, après avoir attendu un nombre spécifié de millisecondes.
- `setInterval(function, milliseconds)`
- Identique à `setTimeout()`, mais répète l'exécution de la fonction en continu.

LA MÉTHODE SETTIMEOUT()

- La `window.setTimeout()` méthode peut être écrite sans le préfixe de `window`.
- Le premier paramètre est une fonction à exécuter.
- Le deuxième paramètre indique le nombre de millisecondes avant l'exécution.

Exemple

Cliquez sur un bouton. Attendez 3 secondes, et la page alertera "Bonjour":

```
<button onclick="setTimeout(myFunction, 3000)">Try it</button>

<script>
function myFunction() {
    alert('Hello');
}
</script>
```


COMMENT ARRÊTER L'EXÉCUTION ?

- La `clearTimeout()` méthode arrête l'exécution de la fonction spécifiée dans `setTimeout()`.
- La `clearTimeout()` méthode utilise la variable renvoyée par `setTimeout()`:

```
myVar = setTimeout(function, milliseconds);  
clearTimeout(myVar);
```

- Si la fonction n'a pas encore été exécutée, vous pouvez arrêter l'exécution en appelant la `clearTimeout()` méthode :
- `<button onclick="myVar = setTimeout(myFunction, 3000)">Try it</button>`
`<button onclick="clearTimeout(myVar)">Stop it</button>`

LA MÉTHODE SETINTERVAL()

- La setInterval() méthode répète une fonction donnée à chaque intervalle de temps donné.
- Le premier paramètre est la fonction à exécuter.
- Le deuxième paramètre indique la longueur de l'intervalle de temps entre chaque exécution.
- Cet exemple exécute une fonction appelée "myTimer" une fois par seconde (comme une montre numérique).

```
<p id="demo"></p>

<script>
setInterval(myTimer, 1000);

function myTimer() {
  const d = new Date();
  document.getElementById("demo").innerHTML = d.toLocaleTimeString();
}
</script>
```

COMMENT ARRÊTER L'EXÉCUTION ?

- La clearInterval() méthode arrête les exécutions de la fonction spécifiée dans la méthode setInterval().
- La clearInterval() méthode utilise la variable renvoyée par setInterval():

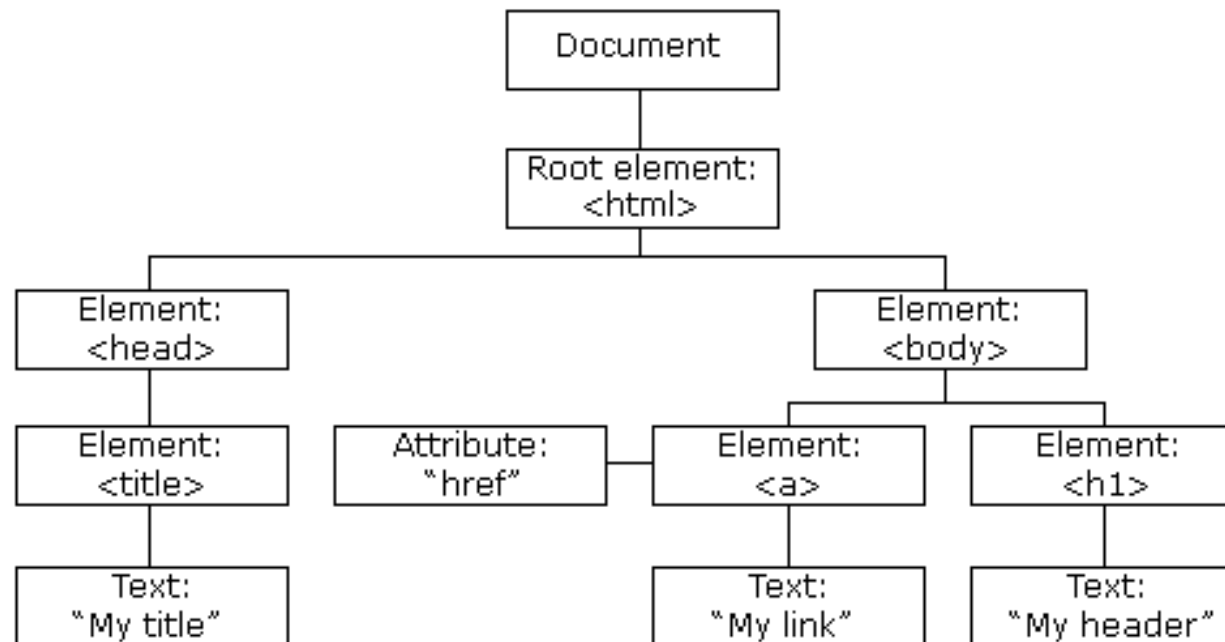
```
<p id="demo"></p>

<button onclick="clearInterval(myVar)">Stop time</button>

<script>
let myVar = setInterval(myTimer, 1000);
function myTimer() {
  const d = new Date();
  document.getElementById("demo").innerHTML = d.toLocaleTimeString();
}
</script>
```

JAVASCRIPT HTML DOM

- Avec le DOM HTML, JavaScript peut accéder et modifier tous les éléments d'un document HTML
- Lorsqu'une page Web est chargée, le navigateur crée un modèle **d' objet** de **document** de la page.
- Le modèle **HTML DOM** est construit comme un arbre d' **Objets** :



- 
- Avec le modèle objet, JavaScript obtient toute la puissance dont il a besoin pour créer du HTML dynamique :
 - JavaScript peut modifier tous les éléments HTML de la page
 - JavaScript peut modifier tous les attributs HTML de la page
 - JavaScript peut modifier tous les styles CSS de la page
 - JavaScript peut supprimer des éléments et des attributs HTML existants
 - JavaScript peut ajouter de nouveaux éléments et attributs HTML
 - JavaScript peut réagir à tous les événements HTML existants dans la page
 - JavaScript peut créer de nouveaux événements HTML dans la page
 -

QU'EST-CE QUE LE DOM HTML ?

- Le DOM HTML est un modèle d' **objet** standard et **une interface de programmation** pour HTML. Il définit :
 - Les éléments HTML en tant **qu'objets**
 - Les **propriétés** de tous les éléments HTML
 - Les **méthodes** pour accéder à tous les éléments HTML
 - Les **événements** pour tous les éléments HTML
- En d'autres termes : **le DOM HTML est une norme sur la manière d'obtenir, de modifier, d'ajouter ou de supprimer des éléments HTML.**

JAVASCRIPT - MÉTHODES HTML DOM

- Les méthodes HTML DOM sont des **actions** que vous pouvez effectuer (sur les éléments HTML).
- Les propriétés HTML DOM sont des **valeurs** (des éléments HTML) que vous pouvez définir ou modifier.
- Le DOM HTML est accessible avec JavaScript (et avec d'autres langages de programmation).
- Dans le DOM, tous les éléments HTML sont définis comme des **objets** .
- L'interface de programmation est constituée des propriétés et des méthodes de chaque objet.
- Une **propriété** est une valeur que vous pouvez obtenir ou définir (comme changer le contenu d'un élément HTML).
- Une **méthode** est une action que vous pouvez effectuer (comme ajouter ou supprimer un élément HTML).

Dans l'exemple ci-dessus, `getElementById` est une méthode, tandis que `innerHTML` est une propriété.

La méthode `getElementById`
Le moyen le plus courant d'accéder à un élément HTML est d'utiliser l'id de l'élément.

Dans l'exemple ci-dessus, la `getElementById` méthode utilisée `id="demo"` pour trouver l'élément.

La propriété `innerHTML`
Le moyen le plus simple d'obtenir le contenu d'un élément consiste à utiliser la `innerHTML` propriété.

La `innerHTML` propriété est utile pour obtenir ou remplacer le contenu des éléments HTML.

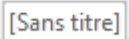
Exemple

```
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```



DOCUMENT HTML DOM

- L'objet de document HTML DOM est le propriétaire de tous les autres objets de votre page Web.
- L'objet document représente votre page Web.
- Si vous souhaitez accéder à n'importe quel élément d'une page HTML, vous commencez toujours par accéder à l'objet document.
- Vous trouverez ci-dessous quelques exemples d'utilisation de l'objet document pour accéder au code HTML et le manipuler.

RECHERCHE D'ÉLÉMENTS HTML

Method	Description
<code>document.getElementById(id)</code>	Trouver un élément par élément ID
<code>document.getElementsByTagName(name)</code>	Trouver des éléments par nom de balise
<code>document.getElementsByClassName(name)</code>	Trouver des éléments par nom de classe

MODIFICATION DES ÉLÉMENTS HTML

Property	Description
<code>element.innerHTML = new html content</code>	Changer le HTML intérieur d'un élément
<code>element.attribute = new value</code>	Modifier la valeur d'attribut d'un élément HTML
<code>element.style.property = new style</code>	Changer le style d'un élément HTML
Method	
	La description
<code>element.setAttribute(attribute, value)</code>	Modifier la valeur d'attribut d'un élément HTML

AJOUT ET SUPPRESSION D'ÉLÉMENTS

Method	Description
<code>document.createElement(element)</code>	Créer un élément HTML
<code>document.removeChild(element)</code>	Supprimer un élément HTML
<code>document.appendChild(element)</code>	Ajouter un élément HTML
<code>document.replaceChild(new, old)</code>	Remplacer un élément HTML
<code>document.write(text)</code>	Écrivez dans le flux de sortie HTML

AJOUT DE GESTIONNAIRES D'ÉVÉNEMENTS

Method	Description
<code>document.getElementById(id).onclick = function(){code}</code>	Ajout du code du gestionnaire d'événements à un événement OnClick

RECHERCHE D'OBJETS HTML

Property	Description
document.anchors	Renvoie tous les <a> éléments qui ont un attribut de nom
document.applets	Décousu
document.baseURI	Renvoie l'uri de base absolue du document
document.body	Renvoie l'élément <body>
document.cookie	Renvoie le cookie du document
document.doctype	Renvoie le doctype du document
document.documentElement	Renvoie l'élément <html>
document.documentMode	Renvoie le mode utilisé par le navigateur
document.documentURI	Renvoie l'URI du document
document.domain	Renvoie le nom de domaine du serveur de documents
document.domConfig	Obsolète.
document.embeds	Renvoie tous les éléments <embed>
document.forms	Renvoie tous les éléments <form>

document.head	Renvoie l'élément <head>
document.images	Renvoie tous les éléments
document.implementation	Renvoie l'implémentation DOM
document.inputEncoding	Renvoie l'encodage du document (jeu de caractères)
document.lastModified	Renvoie la date et l'heure de mise à jour
document.links	Renvoie tous les éléments <a> qui ont un attribut HREF
document.readyState	Renvoie l'état (de chargement) du document
document.referrer	Renvoie l'URI du référentiel (le document de liaison)
document.scripts	Renvoie tous les éléments <script>
document.strictErrorChecking	Renvoie si la vérification des erreurs est appliquée
document.title	Renvoie l'élément <title>
document.URL	Renvoie l'URL complète du document

RECHERCHE D'ÉLÉMENTS HTML

- Souvent, avec JavaScript, vous souhaitez manipuler des éléments HTML.
- Pour ce faire, vous devez d'abord trouver les éléments. Il y a plusieurs moyens de le faire:
- Recherche d'éléments HTML par identifiant
- Recherche d'éléments HTML par nom de balise
- Recherche d'éléments HTML par nom de classe
- Recherche d'éléments HTML par des sélecteurs CSS
- Recherche d'éléments HTML par collections d'objets HTML

RECHERCHE D'ÉLÉMENT HTML PAR IDENTIFIANT

- Le moyen le plus simple de trouver un élément HTML dans le DOM consiste à utiliser l'identifiant de l'élément.
- Cet exemple trouve l'élément avec id="intro":

```
<p id="intro">Finding HTML Elements by Id</p>
<p>This example demonstrates the <b>getElementsById</b> method.</p>

<p id="demo"></p>

<script>
const element = document.getElementById("intro");

document.getElementById("demo").innerHTML =
"The text from the intro paragraph is: " + element.innerHTML;

</script>
```

RECHERCHE D'ÉLÉMENTS HTML PAR NOM DE BALISE

- Cet exemple trouve l'élément avec id="main", puis trouve tous les <p>éléments à l'intérieur "main":

```
<div id="main">
  <p>Finding HTML Elements by Tag Name</p>
  <p>This example demonstrates the <b>getElementsByName</b> method.</p>
</div>

<p id="demo"></p>

<script>
const x = document.getElementById("main");
const y = x.getElementsByTagName("p");

document.getElementById("demo").innerHTML =
  'The first paragraph (index 0) inside "main" is: ' + y[0].innerHTML;

</script>
```

RECHERCHE D'ÉLÉMENTS HTML PAR NOM DE CLASSE

- `const x = document.getElementsByClassName("intro");`

RECHERCHE D'ÉLÉMENTS HTML PAR DES SÉLECTEURS CSS

- Si vous voulez trouver tous les éléments HTML qui correspondent à un sélecteur CSS spécifié (id, noms de classe, types, attributs, valeurs des attributs, etc.), utilisez la `querySelectorAll()` méthode.
- Cet exemple renvoie une liste de tous les `<p>` éléments avec `class="intro"`.

```
<p>Finding HTML Elements by Query Selector</p>
<p class="intro">Hello World!.</p>
<p class="intro">This example demonstrates the <b>querySelectorAll</b> method.</p>

<p id="demo"></p>

<script>
const x = document.querySelectorAll("p.intro");
document.getElementById("demo").innerHTML =
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;
</script>
```

MODIFICATION DE LA VALEUR D'UN ATTRIBUT

- Pour modifier la valeur d'un attribut HTML, utilisez cette syntaxe :

```
document.getElementById(id).attribute = new value
```

```
<!DOCTYPE html>
<html>
<body>



<script>
document.getElementById("myImage").src = "landscape.jpg";
</script>

</body>
</html>
```


FORMULAIRES JAVASCRIPT

- La validation du formulaire HTML peut être effectuée par JavaScript.
- Si un champ de formulaire (fname) est vide, cette fonction alerte un message et renvoie false pour empêcher la soumission du formulaire :

```
<head>
  <script>
    function validateForm() {
      let x = document.forms["myForm"]["fname"].value;
      if (x == "") {
        alert("Name must be filled out");
        return false;
      }
    }
  </script>
</head>

<body>

  <h2>JavaScript Validation</h2>
  <form name="myForm" onsubmit="validateForm()" method="post">
    Name: <input type="text" name="fname">
    <input type="submit" value="Submit">
  </form>

</body>
```

Required suffit pour faire
cette validation

JAVASCRIPT PEUT VALIDER UNE ENTRÉE NUMÉRIQUE

- JavaScript est souvent utilisé pour valider une entrée numérique :

```
<p>Please input a number between 1 and 10:</p>

<input id="numb">
<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
```

MODIFICATION DU STYLE HTML

Pour changer le style d'un élément HTML, utilisez cette syntaxe :

```
document.getElementById(id).style.property = new style
```

L'exemple suivant modifie le style d'un `<p>` élément :

Exemple

```
<html>
<body>

<p id="p2">Hello World!</p>

<script>
document.getElementById("p2").style.color = "blue";
</script>

</body>
</html>
```

Exemple

```
<!DOCTYPE html>
<html>
<body>

<h1 id="id1">My Heading 1</h1>

<button type="button"
onclick="document.getElementById('id1').style.color = 'red'">
Click Me!</button>

</body>
</html>
```

ANIMATION JAVASCRIPT HTML DOM EXERCICE

- Pour montrer comment créer des animations HTML avec JavaScript, nous allons utiliser une simple page Web :

```
<!DOCTYPE html>
<html>
<body>

<h1>My First JavaScript Animation</h1>

<div id="animation">My animation will go here</div>

</body>
</html>
```

Toutes les animations doivent être relatives à un élément conteneur.

```
<div id="container">
  <div id="animate">My animation will go here</div>
</div>
```

- L'élément conteneur doit être créé avec style=" position: relative".
- L'élément d'animation doit être créé avec style=" position: absolute".

```
<!Doctype html>
<html>
<style>
    #container {
        width: 400px;
        height: 400px;
        position: relative;
        background: yellow;
    }

    #animate {
        width: 50px;
        height: 50px;
        position: absolute;
        background: red;
    }
</style>

<body>

    <h2>My First JavaScript Animation</h2>

    <div id="container">
        <div id="animate"></div>
    </div>

</body>

</html>
```

- Les animations JavaScript sont réalisées en programmant des changements progressifs dans le style d'un élément.
- Les modifications sont appelées par une minuterie. Lorsque l'intervalle de minuterie est petit, l'animation semble continue.
- Le code de base est :

```
id = setInterval(frame, 5);

function frame() {
    if (/* test for finished */) {
        clearInterval(id);
    } else {
        /* code to change the element style */
    }
}
```

solution

```
<!DOCTYPE html>
<html>
<style>
  #container {
    width: 400px;
    height: 400px;
    position: relative;
    background: yellow;
  }

  #animate {
    width: 50px;
    height: 50px;
    position: absolute;
    background-color: red;
  }
</style>
```

```
<body>

  <p><button onclick="myMove()">Click Me</button></p>

  <div id="container">
    <div id="animate"></div>
  </div>

  <script>
    function myMove() {
      let id = null;
      const elem = document.getElementById("animate");
      let pos = 0;
      clearInterval(id);
      id = setInterval(frame, 5);

      function frame() {
        if (pos == 350) {
          clearInterval(id);
        } else {
          pos++;
          elem.style.top = pos + "px";
          elem.style.left = pos + "px";
        }
      }
    }
  </script>

</body>

</html>
```


ÉCOUTEUR D'ÉVÉNEMENTS JAVASCRIPT HTML DOM

- La `addEventListener()` méthode attache un gestionnaire d'événements à l'élément spécifié.
- La `addEventListener()` méthode attache un gestionnaire d'événements à un élément sans écraser les gestionnaires d'événements existants.
- Vous pouvez ajouter plusieurs gestionnaires d'événements à un élément.
- Vous pouvez ajouter plusieurs gestionnaires d'événements du même type à un élément, c'est-à-dire deux événements "clic".
- Vous pouvez ajouter des écouteurs d'événements à n'importe quel objet DOM, pas seulement aux éléments HTML. c'est-à-dire l'objet fenêtre.
- La `addEventListener()` méthode facilite le contrôle de la réaction de l'événement au bouillonnement.
- Lors de l'utilisation de la `addEventListener()` méthode, le JavaScript est séparé du balisage HTML, pour une meilleure lisibilité et vous permet d'ajouter des écouteurs d'événement même lorsque vous ne contrôlez pas le balisage HTML.
- Vous pouvez facilement supprimer un écouteur d'événement en utilisant la `removeEventListener()` méthode.

- Il existe deux manières de propager les événements dans le DOM HTML, le *bouillonnement* et la *capture*.
- La propagation des événements est un moyen de définir l'ordre des éléments lorsqu'un événement se produit. Si vous avez un élément `<p>` à l'intérieur d'un élément `<div>` et que l'utilisateur clique sur l'élément `<p>`, quel événement "click" de l'élément doit être géré en premier ?
- Dans le *bouillonnement*, l'événement de l'élément le plus interne est géré en premier, puis l'événement externe : l'événement click de l'élément `<p>` est géré en premier, puis l'événement click de l'élément `<div>`.
- Lors de la *capture*, l'événement de l'élément le plus externe est traité en premier, puis l'événement interne : l'événement click de l'élément `<div>` sera traité en premier, puis l'événement click de l'élément `<p>`.
- Avec la méthode `addEventListener()` vous pouvez spécifier le type de propagation en utilisant le paramètre "useCapture":

```
addEventListener(event, function, useCapture);
```

La valeur par défaut est `false`, qui utilisera la propagation par bulles, lorsque la valeur est définie sur `true`, l'événement utilise la propagation par capture.

```
<!DOCTYPE html>
<html>

<body>

  <h2>JavaScript addEventListener()</h2>

  <p>Click the button to perform a calculation.</p>

  <button id="myBtn">Try it</button>

  <p id="demo"></p>

  <script>
    let p1 = 5;
    let p2 = 7;
    document.getElementById("myBtn").addEventListener("click", function() {
      myFunction(p1, p2);
    });

    function myFunction(a, b) {
      document.getElementById("demo").innerHTML = a * b;
    }
  </script>

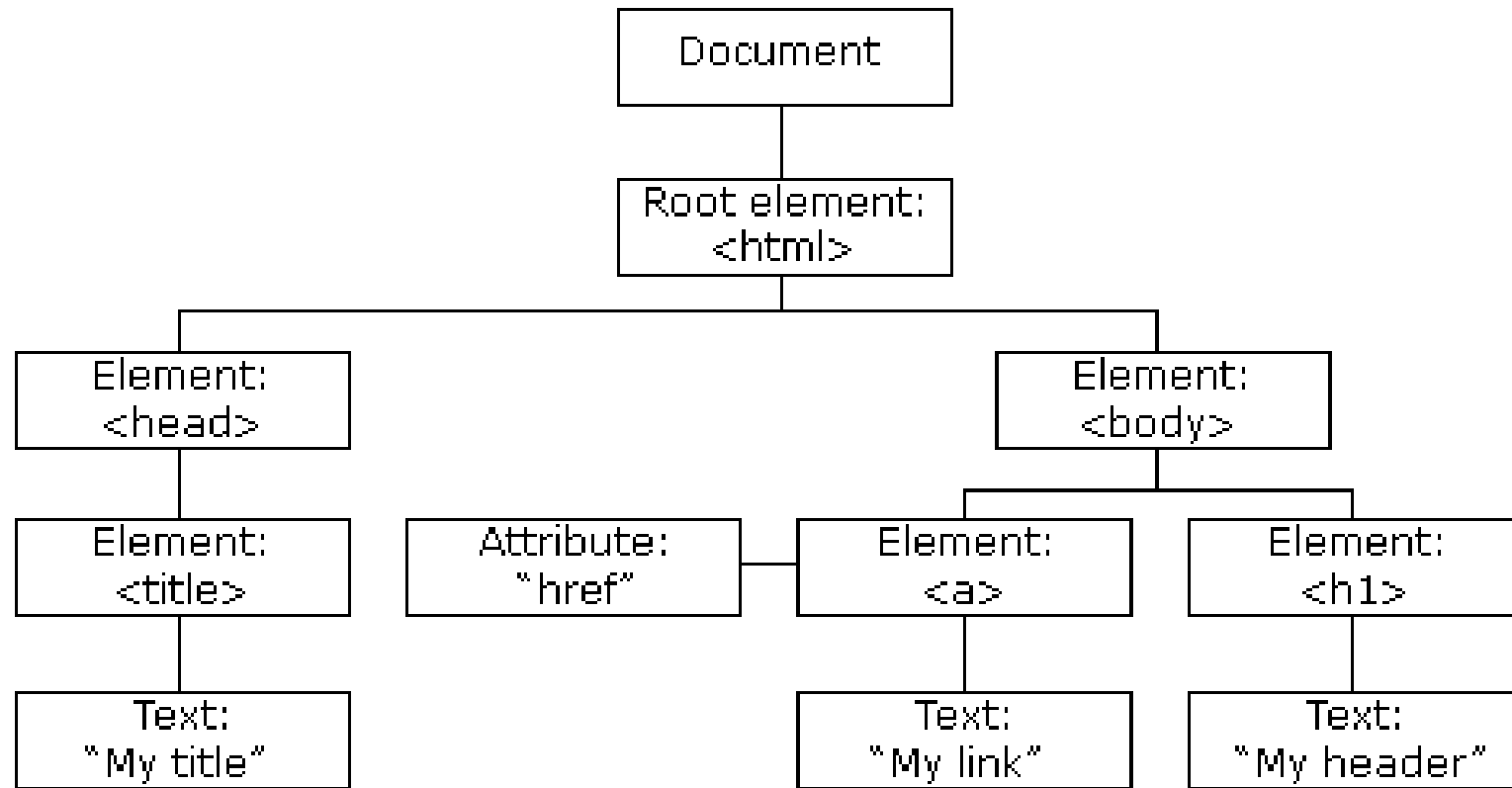
</body>

</html>
```

EXERCICE

NAVIGATION JAVASCRIPT HTML DOM

- Avec le DOM HTML, vous pouvez naviguer dans l'arborescence des nœuds à l'aide des relations entre les nœuds.
- Nœuds DOM
- Selon la norme W3C HTML DOM, tout dans un document HTML est un nœud :
 - Le document entier est un nœud de document
 - Chaque élément HTML est un nœud d'élément
 - Le texte à l'intérieur des éléments HTML sont des nœuds de texte
 - Chaque attribut HTML est un nœud d'attribut (obsolète)
 - Tous les commentaires sont des nœuds de commentaires



- Avec le DOM HTML, tous les nœuds de l'arborescence des nœuds sont accessibles par JavaScript.
- De nouveaux nœuds peuvent être créés et tous les nœuds peuvent être modifiés ou supprimés.

RELATIONS DE NŒUD

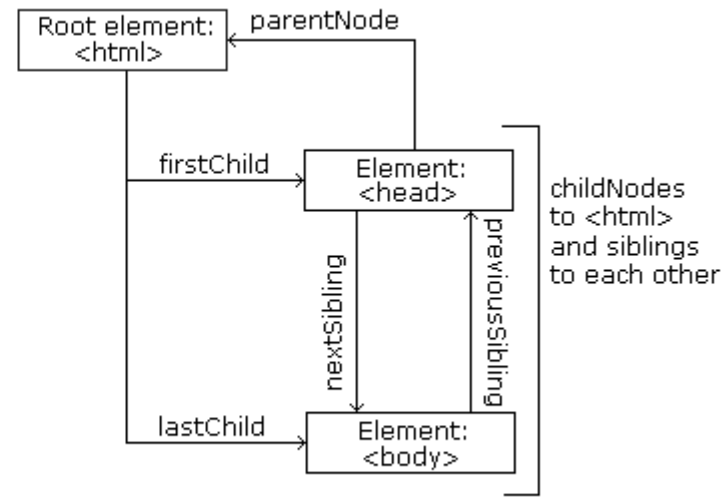
- Les nœuds de l'arborescence des nœuds ont une relation hiérarchique les uns avec les autres.
- Les termes parent, enfant et frère sont utilisés pour décrire les relations.
- Dans une arborescence de nœuds, le nœud supérieur est appelé la racine (ou nœud racine)
- Chaque nœud a exactement un parent, sauf la racine (qui n'a pas de parent)
- Un nœud peut avoir plusieurs fils
- Les frères et sœurs (frères ou sœurs) sont des nœuds avec le même parent

```
<html>

<head>
  <title>DOM Tutorial</title>
</head>

<body>
  <h1>DOM Lesson one</h1>
  <p>Hello world!</p>
</body>

</html>
```



À partir du HTML ci-dessus, vous pouvez lire :

- <html>est le nœud racine
- <html>n'a pas de parents
- <html>est le parent de <head>et<body>
- <head>est le premier enfant de<html>
- <body>est le dernier enfant de<html>

et:

- <head>a un enfant :<title>
- <title>a un enfant (un nœud de texte) : "Tutoriel DOM"
- <body>a deux enfants : <h1>et<p>
- <h1>a un enfant : "DOM Lesson one"
- <p>a un enfant: "Hello world!"
- <h1>et <p>sont frères et soeurs

NAVIGUER ENTRE LES NŒUDS

- Vous pouvez utiliser les propriétés de nœud suivantes pour naviguer entre les nœuds avec JavaScript :
 - parentNode
 - childNodes[nodenumber]
 - firstChild
 - lastChild
 - nextSibling
 - previousSibling

L'accès à la propriété innerHTML est identique à l'accès au nodeValue

```
<!DOCTYPE html>
<html>

<body>

    <h1 id="id01">My First Page</h1>
    <p id="id02"></p>

    <script>
        document.getElementById("id02").innerHTML =
document.getElementById("id01").childNodes[0].nodeValue;
    </script>

</body>

</html>
```

LA PROPRIÉTÉ NODENAME

- La nodeName propriété spécifie le nom d'un nœud.
 - nodeName est en lecture seule
 - nodeName d'un nœud d'élément est le même que le nom de la balise
 - nodeName d'un nœud d'attribut est le nom de l'attribut
 - nodeName d'un nœud de texte est toujours #text
 - nodeName du nœud de document est toujours #document

```
<h1 id="id01">My First Page</h1>
```

```
<p id="id02"></p>
```

```
<script>
```

```
document.getElementById("id02").innerHTML = document.getElementById("id01").nodeName;
```

```
</script>
```

LA PROPRIÉTÉ NODETYPE

La `nodeType` propriété est en lecture seule. Il renvoie le type d'un nœud.

Node	Type	Example
ELEMENT_NODE	1	<code><h1 class="heading">W3Schools</h1></code>
ATTRIBUTE_NODE	2	<code>class = "heading" (deprecated)</code>
TEXT_NODE	3	<code>W3Schools</code>
COMMENT_NODE	8	<code><!-- This is a comment --></code>
DOCUMENT_NODE	9	The HTML document itself (the parent of <code><html></code>)
DOCUMENT_TYPE_NODE	10	<code><!Doctype html></code>

AJOUT ET SUPPRESSION DE NŒUDS (ÉLÉMENTS HTML)

- Création de nouveaux éléments HTML (nœuds)
- Pour ajouter un nouvel élément au DOM HTML, vous devez d'abord créer l'élément (nœud d'élément), puis l'ajouter à un élément existant.

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
  const para = document.createElement("p");
  const node = document.createTextNode("This is new.");
  para.appendChild(node);

  const element = document.getElementById("div1");
  element.appendChild(para);
</script>
```

Exemple expliqué

Ce code crée un nouvel `<p>` élément :

```
const para = document.createElement("p");
```

Pour ajouter du texte à l' `<p>` élément, vous devez d'abord créer un nœud de texte. Ce code crée un nœud de texte :

```
const node = document.createTextNode("This is a new paragraph.");
```

Ensuite, vous devez ajouter le nœud de texte à l' `<p>` élément :

```
para.appendChild(node);
```

Enfin, vous devez ajouter le nouvel élément à un élément existant.

Ce code trouve un élément existant :

```
const element = document.getElementById("div1");
```

Ce code ajoute le nouvel élément à l'élément existant :

```
element.appendChild(para);
```

- Création de nouveaux éléments HTML - insertBefore()
- La appendChild() méthode de l'exemple précédent a ajouté le nouvel élément en tant que dernier enfant du parent.
- Si vous ne le souhaitez pas, vous pouvez utiliser la insertBefore() méthode :

Exemple

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const element = document.getElementById("div1");
const child = document.getElementById("p1");
element.insertBefore(para, child);
</script>
```


- Suppression d'éléments HTML existants
- Pour supprimer un élément HTML, utilisez la remove() méthode :

```
<div>
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const elmnt = document.getElementById("p1"); elmnt.remove();
</script>
```

Exemple expliqué |

Le document HTML contient un `<div>` élément avec deux nœuds enfants (deux `<p>` éléments) :

```
<div>
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>
```

Recherchez l'élément que vous souhaitez supprimer :

```
const elmnt = document.getElementById("p1");
```

Exécutez ensuite la méthode remove() sur cet élément :

```
elmnt.remove();
```

La remove() méthode ne fonctionne pas dans les anciens navigateurs, voir l'exemple ci-dessous pour savoir comment l'utiliser à la removeChild() place.

- Remplacement des éléments HTML
- Pour remplacer un élément dans le DOM HTML, utilisez la `replaceChild()` méthode :

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.replaceChild(para, child);
</script>
```

L'OBJET HTML COLLECTION

- La `getElementsByTagName()` méthode renvoie un `HTMLCollection` objet.
- Un `HTMLCollection` objet est une liste de type tableau (collection) d'éléments HTML.
- Le code suivant sélectionne tous les `<p>` éléments d'un document :

```
<!DOCTYPE html>
<html>

<body>

  <h2>JavaScript HTML DOM</h2>

  <p>Hello World!</p>

  <p>Hello Norway!</p>

  <p id="demo"></p>

  <script>
    const myCollection = document.getElementsByTagName("p");

    document.getElementById("demo").innerHTML = "The innerHTML of the
second paragraph is: " + myCollection[1].innerHTML;
  </script>

</body>

</html>
```

LONGUEUR DE LA COLLECTION HTML

La `length` propriété définit le nombre d'éléments dans un `HTMLCollection` :



Exemple

```
myCollection.length
```

Essayez-le vous-même »

La `length` propriété est utile lorsque vous souhaitez parcourir les éléments d'une collection :

```
<!DOCTYPE html>
<html>

<body>

  <h2>JavaScript HTML DOM</h2>

  <p>Hello World!</p>

  <p>Hello Norway!</p>

  <p>Click the button to change the color of all p elements.</p>

  <button onclick="myFunction()">Try it</button>

  <script>
    function myFunction() {
      const myCollection = document.getElementsByTagName("p");
      for (let i = 0; i < myCollection.length; i++) {
        myCollection[i].style.color = "red";
      }
    }
  </script>

</body>

</html>
```